

Mini Projet DevOps

Implémentation d'un projet en se basant sur une chaîne DevOps de bout en bout

Mohamed Najeh ISSAOUI

E-mail : issaoui.mn@itbs.tn

IT Business School

1. Contexte pédagogique

Dans le cadre du module : ***Pratique DevOps, Chaines d'outils et Automatisation***, vous avez étudié et pratiqué :

- la chaîne DevOps complète (**Plan → Code → Build → Test → Release → Deploy → Operate → Monitor → Improve**)
- l'intégration continue (CI)
- le déploiement continu (CD)
- l'approche GitOps avec Kubernetes & ArgoCD
- la sécurité intégrée (**DevSecOps**)
- l'observabilité (**Monitoring, Logging, Alerting**)

→ Ce mini-projet vise à **valider votre capacité à intégrer TOUTES ces notions dans un projet réel, cohérent et maîtrisé.**

2. Objectif du mini-projet

Vous devez concevoir et implémenter **un projet complet** démontrant votre maîtrise de :

Une chaîne DevOps de bout en bout

→ Ce projet doit être :

- **fonctionnel**
- **automatisé**
- **versionné**
- **déployé**
- **monitoré**
- **sécurisé**

3. Organisation

- Travail **individuel (monôme)**
- Chaque étudiant choisit :
 - son **idée de projet**
 - son **stack technique** (React, Node, Python, etc.)

⚠ Le choix technologique est libre MAIS :

le projet doit permettre de couvrir **toutes les étapes DevOps**

4. Exigence principale

→ Vous devez démontrer :

Votre compréhension réelle de toute la chaîne DevOps à travers votre propre projet

5. Exigences détaillées par phase DevOps

5.1 PLAN — Gestion Agile

Objectif

Montrer que vous savez **organiser un projet comme en entreprise**

Exigences

- Choisir un outil Agile :
 - Jira / Redmine / GitHub Projects / autre
- Définir :
 - backlog produit
 - user stories
 - tâches
- Organiser :
 - sprint(s)
 - workflow (To Do / In Progress / Done)

Livrable attendu

- capture de l'outil Agile
- exemples de user stories

5.2 CODE — Organisation & Git

Objectif

Montrer une **bonne discipline de développement**

Exigences

- Structure claire du projet :

```
frontend/  
backend/  
docker/  
k8s/  
.github/workflows/
```

- utilisation avancée de Git :
 - branches :

```
feat/nom-feature  
bugfix/...  
hotfix/...
```

- commits clairs
- merge vers dev puis main

5.3 BUILD LOCAL

Objectif

Valider la qualité AVANT CI

Exigences

- lint (ESLint ou équivalent)
- tests (unitaires minimum)
- build application
- création image Docker

5.4 VERSIONNING & GITHUB

Exigences

- projet complet sur GitHub
- historique Git propre
- README professionnel :
 - description
 - architecture
 - instructions

5.5 CI — Intégration Continue

Objectif

Automatiser la qualité

Exigences

Créer :

`.github/workflows/ci.yml`

Le pipeline doit contenir :

- checkout
- installation dépendances
- lint
- tests
- build
- analyse qualité :
 - SonarQube (obligatoire)
- build Docker
- push image

→ Le pipeline doit :

- échouer en cas d'erreur
- être clairement structuré

5.6 CD — Déploiement Continu

Objectif

Automatiser le déploiement

Exigences

- Kubernetes (Minikube ou cluster)
- manifests :
 - Deployment
 - Service
- GitOps :
 - Repo ou dossier k8s
- ArgoCD :
 - Synchronisation automatique

→ Aucune commande manuelle de déploiement autorisée après setup

5.7 DEVSECOPS — Sécurité

Objectif

Intégrer la sécurité dans le pipeline

Exigences

- scan code (lint sécurité)
- scan dépendances
- scan image Docker (Trivy)
- gestion des secrets (GitHub Secrets)

→ pipeline doit échouer si vulnérabilité critique

5.8 MONITORING & OBSERVABILITY

Objectif

Superviser l'application

Exigences

- Prometheus :
 - collecte métriques
- Grafana :
 - dashboard
- métriques application (endpoint `/metrics`)
- alerting simple

5.9 IMPROVE — Amélioration continue

Objectif

Montrer une réflexion DevOps

Exigences

- identifier :
 - points faibles
 - améliorations possibles
- proposer :
 - optimisations CI/CD
 - amélioration performance
 - amélioration sécurité

6. Livrables obligatoires

6.1 Repository GitHub

À partager, AVANT de passer à la soutenance, avec :

Mohamed Najeh ISSAOUI

✉ issaoui.mn@itbs.tn

Le repo doit contenir :

- code source
- Dockerfile
- Kubernetes manifests
- pipelines CI/CD
- README professionnel

6.2 Document technique (MAX 10 pages)

⚠ TRÈS IMPORTANT :

✗ Interdit :

- définitions DevOps
- cours théorique
- blabla

✓ Obligatoire :

- description du projet
- architecture
- choix techniques
- explication des pipelines
- captures réelles
- problèmes rencontrés
- solutions apportées

À envoyer, AVANT de passer à la soutenance, à :

- **Mohamed Najeh ISSAOUI**
✉ issaoui.mn@itbs.tn

6.3 Soutenance (20 minutes)

Structure recommandée

1. Présentation du projet (2 min)
2. Architecture DevOps (3 min)
3. Démo pipeline CI (5 min)
4. Démo CD (GitOps + ArgoCD) (5 min)
5. Monitoring (3 min)
6. Amélioration (2 min)

⚠ Règle essentielle

✗ Pas de cours

✗ Pas de définitions

✓ Que du concret basé sur votre projet

7. Grille d'évaluation

Critère	Points
Plan Agile	2
Git & organisation	2
CI pipeline	4
CD (GitOps + Kubernetes)	4
DevSecOps	3
Monitoring	3
Qualité globale	2
Total	20

8. Erreurs éliminatoires

- pipeline CI absent
- déploiement manuel
- pas de Docker
- pas de Kubernetes
- pas de monitoring
- projet non fonctionnel

9. Message final aux étudiants

Ce projet n'évalue pas votre capacité à coder
mais votre capacité à industrialiser un projet
logiciel

10. Niveau attendu

- Niveau ingénieur
- Vision système complète
- Compréhension réelle (pas copier-coller)